TOURISE 2027 — Partner API

Request an Invitation & Newsletter · integration guide for partner websites

Two endpoints behind one key: file an **invitation request**, and **subscribe** someone to the newsletter. This lets your website host its own forms and send the results to TOURISE. A request filed this way lands in the same review queue as one filed on the TOURISE site, under the category you were given.

It is a single JSON `POST`. There is no SDK to install and nothing to embed.

Three things TOURISE gives you

API base URL	`https://<api-host>/api`
Category slug	`e.g. press`
API key	`tri_xxxxxxxxxxxxxxxxxxxxxxxxxxxx`

The key is shown once when it is issued. If it is lost, TOURISE issues a new one — that is also how it is rotated.

"We only have a frontend — will this work?"

Yes. It is a plain `fetch()` and needs no backend of your own. But an API key placed in browser JavaScript is **visible to anyone who opens DevTools**, so pick one of these three deliberately.

A · Call it from a small server-side function RECOMMENDED

Your form posts to your own endpoint; that endpoint adds the key and forwards to TOURISE. One file on Vercel, Netlify, Cloudflare Workers or any host you already use. The key never reaches the browser, and you get no CORS setup at all.

B · Put the key in the browser ACCEPTABLE, WITH EYES OPEN

Works today. Treat the key as public and understand the blast radius:

- The key can **only create a pending request**. It cannot read, list, edit, accept or reject anything, and it can reach no other part of the system.
- It is rate limited to **60 requests per minute per key**.
- Worst case if it is copied: junk requests in the review queue. TOURISE revokes the key and issues a new one.

Requires your exact origin to be added to the API's allow-list — see **CORS**.

C · Ask TOURISE for the reCAPTCHA route instead NO KEY AT ALL

A separate public endpoint takes a Google reCAPTCHA token instead of a key, which suits a browser and holds no secret. It needs TOURISE to whitelist your domain and share their site key. Ask if you would rather ship nothing secret.

Step 1 — Read the field list

`GET` `/api/request-an-invitation/{slug}` · public, no key needed

Tells you which fields to render and which to require, so your form follows the category without a redeploy when TOURISE changes it.

```
{
  "status": "success",
  "data": {
    "slug": "press",
    "name_en": "Press",
    "mandatory_fields": ["first_name", "company"],
    "optional_fields": ["phone"]
  }
}
```

`null` for both lists means the category is not configured — nothing beyond `email` is required.

Step 2 — File the request

`POST` `/api/request-an-invitation/{slug}`

```
POST /api/request-an-invitation/press
X-Api-Key: tri_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Content-Type: application/json

{
  "email": "sara@example.com",
  "first_name": "Sara",
  "last_name": "Idris",
  "company": "Partner Co",
  "job_title": "Editor"
}
```

`Authorization: Bearer <key>` is accepted as the same credential if that header is easier for your HTTP client.

Success

```
HTTP 201
{ "status": "success", "data": { "id": "01a0…", "duplicate": false } }
```

Fields

`email` is always required. Everything else is required only if the category says so in Step 1. **Any field not on this list is ignored** — the API accepts exactly these.

Field	Type	Notes
`email`	string	**Always required.** Identifies the request; duplicates are matched on it.
`first_name`	string	max 100
`last_name`	string	max 100
`company`	string	max 100
`job_title`	string	max 100
`phone`	string	max 100. Send it with the country code.
`industry`	string	max 100
`country_of_residence`	uuid	**A country UUID, not a name.** See below.
`nationality`	uuid	**A country UUID, not a name.** See below.
`social_media_channel_1`	string	max 255
`social_media_channel_2`	string	max 255
`lang`	string	`en` or `ar`. Defaults to `en`. Sets the language of the reply.
`subscribe_newsletter`	boolean	Newsletter opt-in. Send `false` when the box was shown and left unticked — that is a different fact from never asking.

Countries are UUIDs

`country_of_residence` and `nationality` take the `id` from `GET /api/countries` — not `"Saudi Arabia"`. Fetch that list once and use it to populate your dropdowns.

Responses

Code	Meaning · what to do
201	**Filed.** Show your thank-you.
200	**Already pending** for that address — `duplicate: true`, nothing new created. **This is a success.** Show the same thank-you; it stops a double submit putting one person in the queue twice.
401	Key missing, unknown or revoked. Not retryable — contact TOURISE.
404	No category at that slug. Check your spelling of the slug.
410	The category exists but is **closed**. Stop showing the form and say applications have closed.
422	Validation failed. `errors` names each field — show them against your inputs.
429	Rate limited (60/min per key). Back off and retry.

Treat 200 and 201 the same

Both mean the person is in the queue. Only `4xx` is a failure worth showing the user an error for.

Example — browser (option B)

```
// The key is visible to anyone who opens DevTools. See option A.
const res = await fetch(
  `${API_BASE}/request-an-invitation/press`,
  {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Accept': 'application/json',
      'X-Api-Key': API_KEY,
    },
    body: JSON.stringify({
      email: form.email,
      first_name: form.firstName,
      company: form.company,
    }),
  }
);

const body = await res.json();

// 200 = already pending, 201 = created. Both are a success.
if (res.ok) return showThankYou();

if (res.status === 422) return showFieldErrors(body.errors);
if (res.status === 410) return showClosedMessage();
return showGenericError();
```

Example — server-side proxy (option A)

```
// /api/request-invitation  — Vercel / Netlify / Cloudflare function.
// The key stays here and never reaches the browser.
export default async function handler(req, res) {
  const upstream = await fetch(
    `${process.env.TOURISE_API_BASE}/request-an-invitation/press`,
    {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json',
        'X-Api-Key': process.env.TOURISE_API_KEY,   // server-side only
      },
      body: JSON.stringify(req.body),
    }
  );

  const body = await upstream.json();
  res.status(upstream.status).json(body);
}
```

CORS

Only matters for option B. A browser will refuse the call unless your exact origin is on the API allow-list. Send TOURISE the origins you post from, including scheme and any subdomain:

```
https://www.yourdomain.com
https://yourdomain.com
https://staging.yourdomain.com
```

Posting from your own server (option A) needs none of this — CORS is a browser rule and does not apply.

Before you go live

1. `GET` the field list and confirm your form asks for everything in `mandatory_fields`.
2. Submit once and confirm you get `201`.
3. Submit **the same address again** and confirm you get `200` with `duplicate: true` — and that your UI treats it as success, not an error.
4. Submit with a missing required field and confirm you render the `422` `errors` against the right inputs.
5. Submit with a deliberately wrong key and confirm you get `401` and show a sensible message.
6. Ask TOURISE to confirm the request appeared in the review queue.
7. Option B only: confirm your production origin is on the CORS allow-list.

Good to know

- Submitting is a **request to be reviewed** — not a registration and not a confirmed place. Say so on your form, or applicants will expect a ticket.
- TOURISE records which partner site filed each request, so your submissions are attributed to you automatically. You do not send that.
- The rate limit is per key, so another partner cannot use up your budget.
- Nothing here needs Google reCAPTCHA. That is only for option C.

Newsletter signup

The same key, for a signup box on your site.

`POST` `/api/newsletter/subscribe`

```
POST /api/newsletter/subscribe
X-Api-Key: tri_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Content-Type: application/json

{
  "email": "sara@example.com",
  "first_name": "Sara",
  "last_name": "Idris",
  "company": "Partner Co",
  "lang": "en"
}
```

Only `email` is required. New subscribers join the default list automatically.

Say the right thing back

Success is always `201`. Read `state` — "Thanks for subscribing" shown to someone who signed up months ago reads as though the first time never took.

`state`	Meaning	Say
`subscribed`	New — now on the list	"Thanks for subscribing"
`already_subscribed`	Was already on the list	"You are already subscribed"
`resubscribed`	Had unsubscribed, now back	"Welcome back"

`401` for a missing, unknown or revoked key. `422` if the address is not a valid email.

About the `token` field

`token` comes back **only** when `state` is `subscribed` — a genuinely new address. It is a bearer credential: whoever holds it can read that subscriber and unsubscribe them, so it is never returned for an address that already existed, where the caller may know the address without owning it.

Do not build a flow that needs it back for an existing subscriber.

Attribution

TOURISE records which partner site each subscriber came from, taken from your key rather than anything you send. The **first** site to subscribe an address keeps the attribution — a later signup elsewhere does not rewrite where they originally came from.